

Dart Tutorial 5: Loops in Dart

This chapter explains how to repeat tasks efficiently in Dart using loops. After completing this tutorial, students will understand how to use for, while, do-while, and for-in loops.

1. Introduction to Loops

A **loop** is used to execute a block of code repeatedly until a certain condition is met.

Loops save us from writing the same code multiple times and make programs more efficient.

2. Why We Use Loops in Dart

In real world apps, we often need to repeat actions, such as displaying a list of items or processing data multiple times.

Loops allow us to:

- Repeat a task a specific number of times.
 - Process every item inside a collection.
 - Avoid writing repetitive code manually.
 - Build dynamic and efficient programs.
-

3. The for Loop

The **for** loop is used when we know exactly how many times we want to repeat a task.

```
for (int i = 1; i <= 5; i++) {  
  print(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

In this example:

- `int i = 1` is the starting point.
- `i <= 5` is the condition checked before every loop.
- `i++` increases the value after every loop.

4. The while Loop

The `while` loop is used when we want to repeat a task as long as a condition remains true, without knowing the exact number of repetitions in advance.

```
int count = 1;

while (count <= 5) {
  print(count);
  count++;
}
```

Output:

```
1
2
3
4
5
```

5. The do-while Loop

The `do-while` loop runs the code block at least once before checking the condition.

```
int count = 1;

do {
  print(count);
  count++;
} while (count <= 5);
```

Output:

```
1
2
3
```

```
4
5
```

6. for vs while vs do-while

Feature	for	while	do-while
Condition Checked	Before loop	Before loop	After loop
Best For	Known repetitions	Unknown repetitions	At least one execution required
Runs At Least Once	No	No	Yes

7. The for-in Loop

The **for-in** loop is used to go through each item inside a list or collection.

```
List<String> fruits = ['Apple', 'Banana', 'Mango'];

for (String fruit in fruits) {
    print(fruit);
}
```

Output:

```
Apple
Banana
Mango
```

8. The break Statement

The **break** statement is used to stop a loop immediately, even if the condition is still true.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    print(i);
}
```

Output:

```
1
2
3
4
```

9. The continue Statement

The **continue** statement is used to skip the current loop iteration and move to the next one.

```
for (int i = 1; i <= 5; i++) {
  if (i == 3) {
    continue;
  }
  print(i);
}
```

Output:

```
1
2
4
5
```

10. Conclusion of Tutorial 5

In this tutorial, students have learned:

1. What loops are and why they are important.
2. How to use for, while, and do-while loops.
3. The key differences between all three loop types.
4. How to use for-in to loop through a list.
5. How to control loops using break and continue.

In the next tutorial, we will learn about Strings in Dart.

This completes Dart Tutorial 5.